Cache-Coherent Safety Islands for Low-Latency CPS Policy Monitoring

Project Proposal · Advanced Computer Architecture

Nazmus Shakib Sayom

University of Utah · February 2026

Motivation: Functional Safety Implies Bounded Time

- Functional safety requires detecting and controlling faults before hazards occur.
- The fault-tolerant time interval (FTTI) bounds how long we can take to *detect + react*.
- Modern CPS/automotive SoCs already include a safety island: isolated compute that can supervise the main system and force a fail-safe reaction.

Opportunity

Move beyond watchdogs and fault flags to **online policy monitoring** over real signals.

Safety Islands: Where They Sit and What They Do

- **Isolation.** Separate core(s), memory region, and trusted software stack.
- **Supervision.** Observe system health and safety-relevant signals; decide when to trigger a safe state.
- **Interfaces.** Today, many interfaces are coarse (fault flags, watchdog kicks, periodic status).

Missing Today

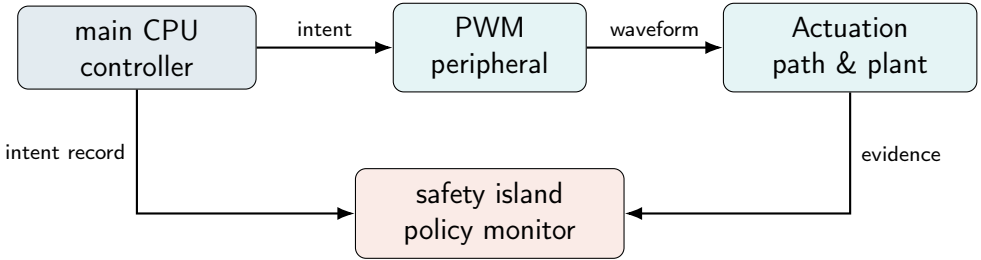
Architecture support to let the safety island check *behavioral safety policies* fast enough to matter.

Policy Monitoring: Intent vs. Evidence

Intent	Evidence
What the controller asked for:	What the system actually did:
▶ Duty cycle, dead-time	▶ Measured PWM edges
▶ Update sequence, timestamp	▶ Waveform timing

- Policies are temporal constraints over signals; STL is one common encoding.
- The safety island correlates them and triggers a reaction if the policy is violated.

Example: PWM Actuation Integrity



- Even correct control software can be undermined by faults/timing interference downstream.
- We need policies that relate *intended* and *realized* actuation with deadlines/dwell times.

State of Practice: DMA/IPC Safety-Island Interfaces

- Most current safety island designs ingest data via **IPC mailboxes** and DMA-friendly shared buffers.
- Examples: Arm Kronos documents HIPC cross-domain messaging; NVIDIA DriveOS describes a functional safety island with controlled channels.
- For step-by-step intent export, these paths often require extra copies or cache maintenance.
- That overhead and jitter competes directly with tight FTTI budgets.

Our Proposal

Use a **small cache-coherent shared region** as the safety-island interface for runtime policy monitoring.

Problem Statement

Check safety policies within FTTI, using intent and evidence that live in different protection domains.

Constraints.

- **Low latency:** the monitor must see new controller intent quickly.
- **Low overhead:** intent export should not burden the control loop.
- **Correctness:** the island must read a *consistent* controller record (not a mixed snapshot).
- **Predictability:** worst-case export + monitoring time must be bounded under contention.

Goal and Hypothesis

Hypothesis

Cache-coherent shared memory is the right substrate for low-latency intent export from the main CPU to an MCU-like safety island monitor.

Project goal. Define the **architecture contract** that makes coherent monitoring both correct and time-bounded:

- coherent interface choice for an MCU-like safety island,
- synchronization discipline for multi-field records,
- predictability mechanisms to meet FTTI.

Why Cache-Coherent Shared Memory?

Goal: export controller intent to the safety island with **minimal latency and overhead**.

- **Single-copy path:** controller writes once; island reads via ordinary loads (no message copying).
- **Compared to IPC/DMA:** avoids extra copies and software-managed cache flush/invalidate on the fast path.
- **Good fit for control:** frequent small records, tight deadlines.

Key Insight

Cache coherence lets both cores observe a single up-to-date value *per memory location* without explicit cache maintenance. This proposal adds **snapshot correctness** and **bounded latency** for the coherent export path.

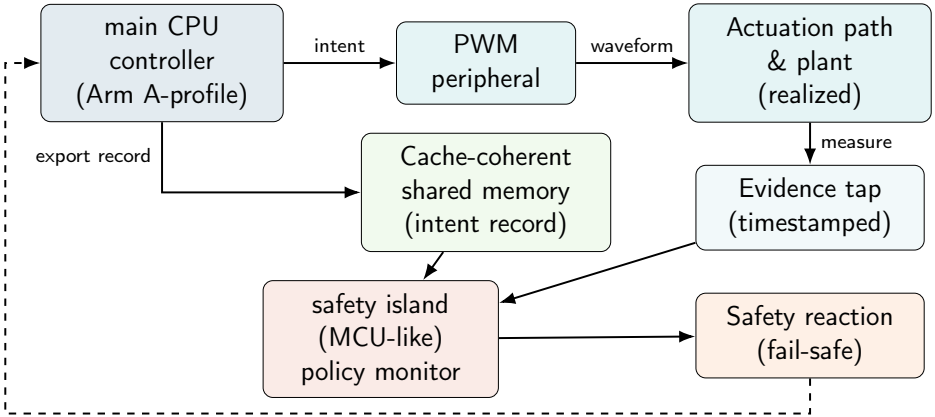
Related Work and Gap

- **Safety islands (practice):** Arm Kronos/HIPC and NVIDIA DriveOS describe isolated islands, typically using IPC/DMA-style channels.
- **Coherent fabrics (standards):** Arm AMBA ACE/CHI provide coherence-capable interconnects and coherent I/O options.
- **Correctness under weak ordering:** Arm memory-model work (e.g., Pulte et al.) and heterogeneous consistency work (e.g., MemGlue, Compound Consistency).
- **Bounded latency under contention:** predictable coherence (e.g., Hassan et al., Kaushik et al.) and mixed-criticality sharing vs. isolation (e.g., Chisholm et al.).
- **Monitoring foundations:** Simplex-style runtime assurance and temporal monitors (e.g., RTAMT), but they often assume the trace is already well-timed.

Gap

An end-to-end architecture for **coherent intent export** + time-aligned evidence + bounded detection latency for safety-island policy monitoring.

Proposed Architecture



Coherence provides a low-latency export path; this project defines the contract that makes it correct under weak ordering and time-bounded under contention.

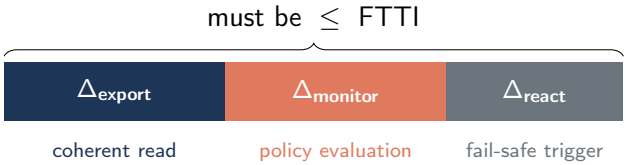
Challenge 1: Consistent Records

Weak ordering: other cores may observe your writes in a different order than your program order (unless you synchronize).

Writer (controller)	Reader (safety island)
`store duty := new` ✓	`load ts == new` ✓
`store ts := new` ✓	`load duty == old` ×
	mixed snapshot!

We need an **export record + synchronization discipline** so island reads behave like consistent "steps."

Challenge 2: Bounded Detection Latency



- **Export:** coherence traffic Time it takes to propagate coherent data.
- **Monitor:** policy evaluation over the current signal window. Bounded by the policy horizon and evaluation complexity.
- **React:** triggering the fail-safe output. Hardware path; typically small and deterministic.

Contention inflates Δ_{export}; we need **predictability mechanisms** (partitioning, priorities, protocol choices) to bound the worst case.

Research Questions

RQ1 **Coherent interface.** How should an MCU-like safety island participate in coherence with an A-profile main CPU?

RQ2 **Snapshot correctness.** What record format + synchronization rules guarantee consistent multi-field snapshots under weak ordering?

RQ3 **Predictability.** What worst-case export and monitoring latency bounds are achievable under contention, and what mechanisms enforce them?

RQ4 **Evidence and time alignment.** Where do we measure realized actuation, and how do we align evidence with controller records?

Case Study: PWM Actuation Integrity

Signals.

- **Intent** (from shared record): duty, dead-time, update sequence, timestamp.
- **Evidence:** edge timings / waveform-derived duty, glitch signatures, timing anomalies.

Example policies (illustrative).

- Tracking: realized duty converges to intended duty within a deadline and stays within tolerance.
- Glitch exclusion: no pulses below a minimum width.
- Safety envelope: upon a safety condition, realized PWM enters a conservative envelope within a deadline.
- Policies can be encoded in STL or similar temporal formalisms; robust monitoring can output a margin.

Tools and Interfaces We Will Explore

- **Coherent interfaces:** Arm AMBA ACE/CHI to study coherent agent vs. coherent I/O attachment and access control for a shared region.
- **Ordering validation:** litmus tests + memory-model tools (e.g., Herd7) to validate the record format and synchronization discipline under weak ordering.
- **Latency analysis:** microbenchmarks and/or simulation (e.g., gem5) to characterize coherent access latency under contention and relate it to FTTI budgets.
- **Monitor prototyping:** a temporal monitoring library (e.g., RTAMT) to prototype policy monitors on PWM traces.

Expected Contributions

- A **cache-coherent** safety-island monitoring architecture for low-latency CPS policy checks.
- A monitoring-oriented **shared-memory contract**: record format, ordering rules, time-alignment assumptions.
- A predictability analysis for coherent intent export under contention.
- A PWM actuation-integrity case study that grounds policies and evidence choices.

Wrap-Up

Takeaway. We push cache-coherent shared memory (not DMA/IPC) as the safety-island interface for low-latency intent export.

- Define a coherent interface + synchronization contract so the island reads consistent controller records.
- Bound worst-case coherent-access latency so policy checks stay meaningful within FTTI.
- Ground the design in a PWM actuation-integrity case study (intent vs. evidence).

Questions?